

UniMediTrend

Alternative Models & Deep-Dive ML Q&A

Why This? Why Not That? — Everything You Need to Defend Your Choices

GROUP 8, CE 3A | UMaT, Tarkwa

Artificial Intelligence in Engineering

Dr. Abdel-Fatao Hamidu

Contents

PART A Alternative Models We Could Have Used

A1 Alternative Clustering Models (beyond K-Means)

A2 Alternative Forecasting / Regression Models (beyond LR & RF)

A3 Summary Comparison Table — All Models

PART B Deep-Dive ML Question & Answer Bank

B1 Why Questions — Defending UniMediTrend Choices

B2 How / What Questions — Concept Mechanics

B3 Tricky / Examiner-Favourite Questions

PART A — Alternative Models We Could Have Used

UniMediTrend used three models: **K-Means** (clustering), **Linear Regression** (forecasting baseline), and **Random Forest** (forecasting). Below are credible alternatives for both tasks — what they are, why they could help, and why we didn't use them.

A1. Alternative Clustering Models

DBSCAN — Density-Based Spatial Clustering of Applications with Noise

How it works: DBSCAN defines clusters as dense regions of points separated by sparse regions. It does NOT require you to specify K in advance. It classifies points as: **core points** (have at least MinPts neighbours within radius epsilon), **border points** (within epsilon of a core point but not core themselves), and **noise/outliers** (neither core nor border).

Why it could have been better for UniMediTrend:

- + Does not require specifying K — discovers the number of clusters automatically.
- + Can find clusters of arbitrary shape (not just spherical like K-Means).
- + Naturally identifies and labels outlier/noise visit records — useful for detecting unusual clinic events.
- + Better suited if hostel risk profiles form irregular non-spherical groupings.

Why we didn't use it / its limitations:

- – Requires tuning two hyperparameters: epsilon (neighbourhood radius) and MinPts (minimum points). Choosing these is non-trivial.
 - – Struggles with high-dimensional, one-hot encoded data (the 'curse of dimensionality' makes density estimation unreliable).
 - – Our visit feature space (Model B) is high-dimensional after encoding — DBSCAN would underperform there.
 - – Does not produce a fixed number of interpretable tiers, which is what clinic managers need.
-

GMM — Gaussian Mixture Models

How it works: GMM assumes data is generated from a mixture of K Gaussian (normal) distributions. Instead of hard cluster assignment (you belong to cluster 1), GMM gives each point a **probability** of belonging to each cluster. It is fit using the **Expectation-Maximization (EM)** algorithm.

Why it could have been better for UniMediTrend:

- + Produces soft/probabilistic cluster assignments — a hostel can be 70% Tier 1, 30% Tier 2. More nuanced than hard K-Means labels.
- + Allows clusters to have different sizes, shapes, and orientations (elliptical, not just spherical).
- + Better handles the natural overlap between visit archetypes in Model B.
- + AIC and BIC criteria can be used to choose the optimal number of components.

Why we didn't use it / its limitations:

- – More computationally expensive than K-Means — requires iterative EM convergence.
 - – The Gaussian assumption may not hold for clinical data which can be skewed or multimodal.
 - – With only 8 hostels (Model A), fitting a GMM is statistically unreliable — too few data points.
 - – Harder to explain to non-technical stakeholders ('70% probability of being high-risk' vs 'Tier 1 hostel').
-

Hierarchical Clustering — Agglomerative / Divisive Hierarchical Clustering

How it works: Agglomerative clustering starts with each point as its own cluster, then repeatedly merges the two closest clusters until one cluster remains. The result is a **dendrogram** — a tree diagram showing how clusters merged. You cut the tree at a chosen height to get your desired number of clusters.

Why it could have been better for UniMediTrend:

- + Does not require specifying K upfront — you choose the cut level after seeing the dendrogram.
- + The dendrogram visually shows the natural hierarchy of hostel relationships — which hostels are most similar.
- + With only 8 hostels (Model A), hierarchical clustering is perfectly sized — K-Means can be unstable with so few points.
- + No centroid assumption — handles non-spherical cluster shapes.

Why we didn't use it / its limitations:

- – Computationally expensive for large datasets — $O(n^2)$ memory. Unsuitable for 5,799 individual visit records (Model B).
 - – Once merged, clusters cannot be un-split (agglomerative is greedy — early merge errors propagate).
 - – Dendrogram interpretation requires domain expertise to choose the right cut point.
-

K-Medoids (PAM) — Partitioning Around Medoids

How it works: Similar to K-Means but instead of using the mean (centroid) of a cluster, K-Medoids uses actual data points as cluster centres (**medoids**). The medoid is the point that minimizes total dissimilarity to all other points in the cluster.

Why it could have been better for UniMediTrend:

- + More robust to outliers than K-Means — a medoid is a real data point, not a synthetic mean.
- + Works with any distance metric (not just Euclidean) — useful for mixed-type clinical data.
- + Actual data points as cluster centres are more interpretable: 'Cluster represented by this specific visit profile'.

Why we didn't use it / its limitations:

- – Computationally much more expensive than K-Means — $O(k * (n-k)^2)$ per iteration.
 - – For 5,799 records (Model B), PAM would be very slow without approximations.
 - – For our synthetic data with clean Euclidean features, K-Means performs similarly at lower cost.
-

A2. Alternative Forecasting / Regression Models

XGBoost — Extreme Gradient Boosting

How it works: XGBoost is a gradient boosting framework. It builds trees **sequentially** — each new tree corrects the errors (residuals) of the previous tree. It minimizes a regularized loss function using gradient descent. XGBoost adds L1/L2 regularization, missing value handling, and parallel computation on top of standard gradient boosting.

Why it could have improved UniMediTrend's forecasting:

- + Handles non-linear relationships between lag features and visit counts much better than Linear Regression.
- + Built-in regularization (lambda, alpha) prevents overfitting — critical given our small 79-row dataset.
- + Provides feature importance scores — reveals which lag or cyclical feature drives demand the most.
- + Consistently outperforms Random Forest on tabular data in practice.
- + Explicitly mentioned in our Future Work section as a planned improvement.

Why we didn't use it / its limitations:

- – More hyperparameters to tune (learning rate, max depth, n_estimators, subsample, colsample_bytree) — requires cross-validated tuning.
 - – With only 79 training rows, gradient boosting can still overfit if not carefully regularized.
 - – Less interpretable than Linear Regression — harder to explain coefficient-level reasoning.
-

LightGBM — Light Gradient Boosting Machine

How it works: LightGBM is Microsoft's gradient boosting framework. It uses two key innovations: **Gradient-Based One-Side Sampling (GOSS)** — keeps instances with large gradients and randomly drops those with small gradients; and **Exclusive Feature Bundling (EFB)** — bundles mutually exclusive sparse features to reduce dimensionality. It grows trees **leaf-wise** (best-first) rather than level-wise.

Why it could have improved UniMediTrend's forecasting:

- + Faster training than XGBoost — especially relevant if dataset grows larger over time.
- + Leaf-wise growth captures more complex patterns than level-wise (Random Forest) growth.
- + Also mentioned in the Future Work section.
- + Handles categorical features natively without one-hot encoding.

Why we didn't use it / its limitations:

- – Leaf-wise growth is more prone to overfitting on small datasets — our 79 rows is a significant risk.
 - – Even faster than XGBoost but more complex to tune well.
-

ARIMA / SARIMA — AutoRegressive Integrated Moving Average (Seasonal variant)

How it works: ARIMA is a classical time-series model. It models the target as a function of: **AR** (AutoRegressive) — past values; **I** (Integrated) — differencing to make the series stationary; **MA** (Moving Average) — past forecast errors. SARIMA adds seasonal components (P, D, Q, s) for periodic patterns.

Why it could have improved UniMediTrend's forecasting:

- + Specifically designed for time-series — handles autocorrelation and trend natively without feature engineering.
- + SARIMA would directly model the academic semester seasonality we observed in EDA.
- + Does not require feature engineering of lag, rolling mean, or cyclical encodings — these are built into the model structure.
- + Well-established statistical framework with interpretable parameters.
- + Works well on short series (79 rows is reasonable for ARIMA).

Why we didn't use it / its limitations:

- – Assumes linearity and stationarity — clinic demand driven by non-linear factors (exam stress, disease outbreaks) may violate this.
 - – Requires stationarity testing (ADF test) and careful order selection (p, d, q) — adds complexity.
 - – Pure time-series model — cannot incorporate external features like hostel cluster labels or diagnosis type.
 - – Does not generalize to a machine learning pipeline framework as cleanly as scikit-learn models.
-

SVR — Support Vector Regression

How it works: SVR applies the Support Vector Machine principle to regression. It tries to fit a hyperplane within an epsilon-tube around the data, minimizing only errors that fall outside this tube. The **kernel trick** (RBF, polynomial, sigmoid) maps data into higher-dimensional spaces to capture non-linear relationships.

Why it could have improved UniMediTrend's forecasting:

- + Handles non-linear patterns via kernel trick — could capture complex academic cycle effects.
- + Effective in high-dimensional spaces and small datasets — suitable for our 79-row problem.
- + Robust to outliers — the epsilon-insensitive loss ignores small errors and focuses on larger deviations.

Why we didn't use it / its limitations:

- – Requires careful tuning of C (regularization), epsilon (tube width), and kernel parameters.
 - – Not interpretable — no feature importance, no coefficient explanation.
 - – Computationally expensive for large datasets (though irrelevant at 79 rows).
 - – Less naturally suited to time-series than ARIMA or lag-based tree models.
-

Ridge / Lasso Regression — Regularized Linear Regression

How it works: Ridge (L2) and Lasso (L1) add penalty terms to the Linear Regression loss function to shrink coefficients. **Ridge:** adds sum of squared coefficients — shrinks all coefficients toward zero but rarely to exactly zero. **Lasso:** adds sum of absolute coefficients — can shrink some coefficients to exactly zero, performing automatic **feature selection**.

Why it could have improved UniMediTrend's forecasting:

- + Our dataset has few rows (79) but several engineered features — regularization prevents overfitting that plain Linear Regression is prone to.
- + Lasso would automatically identify which lag or cyclical features are most predictive, discarding irrelevant ones.
- + Ridge would provide a more stable linear baseline than plain OLS when features are correlated (e.g., lag_1 and rolling_mean_4).
- + Same interpretability as Linear Regression — coefficients are still readable.

Why we didn't use it / its limitations:

- - Still linear — cannot capture non-linear demand spikes during exam periods or disease outbreaks.
 - - With only 7–10 features, the regularization benefit is modest — overfitting risk is already limited.
-

Prophet — Facebook/Meta Prophet

How it works: Prophet is a decomposable time-series model developed by Meta. It models the series as: Trend + Seasonality + Holidays + Noise. It uses a piecewise linear or logistic growth trend, Fourier series for seasonality, and allows explicit holiday/event effects. Designed to work well with daily/weekly data with missing values and outliers.

Why it could have improved UniMediTrend's forecasting:

- + Directly models academic semester patterns as seasonality — no need to manually engineer sin/cos encodings.
- + Handles exam calendar events and public holidays as explicit regressors — exactly the exogenous signals our project lacks.
- + Robust to missing data and outliers — common in clinic records.
- + Produces **uncertainty intervals** — tells the clinic 'we predict 60 visits, but it could range from 45 to 75'.
- + Very easy to use and interpret for non-technical stakeholders.

Why we didn't use it / its limitations:

- - Designed for longer time series (ideally 1+ year of daily data) — our 79 weekly points is on the shorter end.
 - - Less flexible for incorporating engineered ML features (cluster labels, diagnosis proportions).
 - - Not part of the scikit-learn ecosystem — harder to integrate into the existing pipeline.
-

A3. Summary Comparison — All Models

Clustering Models

Model	Needs K?	Handles Noise?	Shape Assumption	Best For
K-Means (USED)	Yes	No	Spherical	Known K, clean features, fast
DBSCAN	No	Yes (labels outliers)	Any shape	Irregular clusters, noise detection
GMM	Yes	Soft boundaries	Elliptical	Overlapping clusters, probabilistic assignment
Hierarchical	No (post-hoc)	No	Any shape	Small datasets, dendrogram insight
K-Medoids	Yes	Better than K-Means	Spherical	Outlier robustness, interpretable centres

Forecasting Models

Model	Non-Linear?	Interpretable?	Small Data?	Uncertainty?	Best For
Linear Regression (USED)	No	High	Good	No	Baseline, interpretability
Random Forest (USED)	Yes	Medium	OK	No	Non-linear tabular data
XGBoost	Yes	Medium	Good*	No	Best tabular accuracy
LightGBM	Yes	Medium	Risk	No	Large data, speed
ARIMA/SARIMA	No	High	Good	No	Pure time-series, seasonality
SVR	Yes (kernel)	Low	Good	No	Small high-dim data
Ridge/Lasso	No	High	Good	No	Regularized linear baseline
Prophet	Partial	High	Medium	Yes	Trend+seasonality+events

* Good with regularization tuned. Without tuning, XGBoost can overfit on 79 rows.

PART B — Deep-Dive ML Question & Answer Bank

These are the questions most likely to be asked in a presentation or viva. Focus especially on the 'Why' questions — these are where marks are won or lost.

B1. 'Why' Questions — Defending UniMediTrend Choices

Q1. Why did you use K-Means and not another clustering algorithm?

K-Means was appropriate for three reasons. First, it is computationally efficient — it scales well to our 5,799 visit records for Model B. Second, we had domain-driven priors: we expected approximately 3 hostel risk tiers and 4 visit archetypes, making specifying K in advance reasonable. Third, our data was preprocessed with StandardScaler and one-hot encoding, producing a relatively clean Euclidean feature space that satisfies K-Means' spherical cluster assumption reasonably well.

The main limitation — that K-Means assumes spherical clusters — is acknowledged. The moderate silhouette score of 0.1120 for Model B confirms some non-spherical overlap, which is why we validated with both the Elbow Method and Silhouette analysis rather than relying on K-Means output blindly.

Q2. Why did you choose K=3 for Model A and K=4 for Model B?

For Model A (hostel risk), K=3 was chosen using both quantitative and qualitative reasoning. The Elbow Method showed diminishing inertia returns beyond K=3 for the hostel-level profile matrix. The Silhouette Score of 0.3772 at K=3 was the highest across tested values. Qualitatively, K=3 maps naturally onto three operationally meaningful intervention categories: malaria control, stress/mental health, and sanitation/food safety.

For Model B (visit archetypes), K=4 was chosen from the Elbow and Silhouette diagnostics shown in Figure 6.1. K=4 also aligns with clinically interpretable categories — tropical illness, waterborne/severe, burnout, and minor ailments — which makes the output actionable for clinic staff.

Q3. Why did you use a synthetic dataset instead of real clinic data?

Health records contain sensitive personal data (name, diagnosis, residential location) which is subject to strict data protection regulations. Using real data without formal ethical clearance and anonymization protocols would be a governance violation.

Synthetic data was generated to preserve statistical complexity — including realistic academic cycles, seasonal disease patterns, and diagnosis distributions — while ensuring no real patient is identifiable. This is a standard approach in healthcare AI research and protects both patients and the research team.

Q4. Why did you use StandardScaler before running K-Means?

K-Means assigns points to clusters based on Euclidean distance to centroids. If features are on different scales — for example, severity ranges from 1–10 while a one-hot encoded hostel column is 0 or 1 — the larger-valued features will dominate the distance calculation, making the clustering effectively ignore the smaller-valued features.

StandardScaler transforms each feature to have mean=0 and standard deviation=1, putting all features on equal footing. Without scaling, K-Means would produce clusters driven almost entirely by the highest-variance features, regardless of their clinical importance.

Q5. Why did you use one-hot encoding instead of label encoding?

Label encoding assigns arbitrary integers to categories (e.g., Malaria=1, Typhoid=2, Cold=3). This implies an ordinal relationship — that Typhoid is 'more' than Malaria — which is clinically meaningless and would mislead both K-Means (which uses distance) and linear regression (which assumes numeric relationships).

One-hot encoding creates a separate binary column for each category, treating all diagnoses as equally distinct with no implied ordering. For distance-based algorithms and linear models, one-hot encoding is the correct approach for nominal categorical variables.

Q6. Why did you use a chronological holdout split instead of random split?

Our target variable is weekly visit counts — a time-ordered sequence. Random splitting would allow the model to train on future data and test on past data, which constitutes look-ahead leakage. The model would appear to perform well because it has already 'seen' future patterns during training.

In real deployment, we only ever know the past. A chronological 80/20 split correctly simulates this: the model trains on weeks 1 through 63 and is tested on weeks 64 through 79 — data it has genuinely never seen. This gives an honest estimate of real-world predictive performance.

Q7. Why did you use 5-Fold TimeSeriesSplit instead of standard k-fold cross-validation?

Standard k-fold cross-validation randomly shuffles data before splitting. For time-series data, this is invalid — it breaks the temporal ordering and allows future information to contaminate training folds, producing optimistically biased performance estimates.

TimeSeriesSplit uses expanding training windows: Fold 1 trains on the first 20% and tests on the next 20%; Fold 2 trains on the first 40% and tests on the next 20%; and so on. This correctly simulates how the model would be used in practice — always predicting forward in time — while also giving us robustness diagnostics across multiple academic sub-periods.

Q8. Why were your R^2 values negative, and does that mean the project failed?

Negative R^2 means the model's predictions were worse than simply predicting the historical mean for every week. For example, if average weekly visits is 65 and we just predicted 65 every week, that naive baseline would have scored $R^2=0$. Our models scored $R^2=-0.18$ and -0.50 , meaning they made the prediction worse in those specific experimental conditions.

This is NOT a project failure — it is a diagnostic finding. The pipeline is correctly engineered: data is clean, features are sound, validation is rigorous. What is missing are exogenous signals. Clinic demand is driven by factors outside the visit records themselves — exam timetables, weather, disease outbreaks. Without these, the model has insufficient information to beat a simple mean. The report is transparent about this limitation, which is itself a mark of scientific integrity.

Q9. Why did you use PCA only for visualization and not as a preprocessing step for clustering?

PCA reduces dimensionality by projecting data onto lower-dimensional principal components. If used before clustering, it would change the feature space that K-Means operates on, which could lose clinically meaningful variance in specific diagnosis columns.

We used PCA only as a post-hoc visualization tool — to project the K=4 cluster results into 2D (PC1 vs PC2) for Figure 6.2. The actual K-Means assignment was done in the full original feature space. This preserves the integrity of the clustering while still allowing us to visually inspect and communicate cluster separation.

Q10. Why did you use lag features instead of just using the raw time index as a feature?

The raw time index (week number 1, 2, 3...) tells the model nothing about the recent clinical context — it only encodes 'how far along in time we are'. It cannot capture the fact that a spike last week often predicts higher demand this week.

Lag features (lag_1, lag_2, lag_3) give the model short-term memory of actual visit volumes. If last week had 100 visits, the model learns this likely precedes a continued high-demand period. If last week had 5 visits (semester break), the model learns demand is likely still low. This is a form of autoregressive modeling — encoding the signal's own recent history as predictive information.

Q11. Why did you use rolling_mean_4 as a feature?

Individual weekly visit counts are noisy — a single week might spike due to a food poisoning event that resolves the following week. A single lag feature would then train the model to expect continued high visits when the spike was in fact temporary.

The 4-week rolling mean smooths out these short-term fluctuations and gives the model a view of the prevailing demand trend rather than instantaneous noise. It captures whether the general trend is upward, downward, or stable — a cleaner signal for predicting the next week's baseline demand.

B2. 'How / What' Questions — Concept Mechanics

Q1. What is the difference between supervised and unsupervised learning? Give examples from UniMediTrend.

In supervised learning, the model is trained on labelled examples — pairs of (input, correct output). The model learns to map inputs to outputs by minimizing prediction error. In UniMediTrend, forecasting is supervised: the input is a feature vector (lag_1, rolling_mean_4, month_sin...) and the label is the known total_visits for that week. The model learns the mapping.

In unsupervised learning, there are no labels — the model must discover structure in the data entirely on its own. In UniMediTrend, clustering is unsupervised: we give K-Means the hostel profile matrix or visit feature vectors with no pre-assigned risk tiers. K-Means discovers the tiers by finding natural groupings based on similarity.

Q2. What is the Elbow Method and how does it work mathematically?

The Elbow Method plots inertia against increasing values of K (typically K=2 to K=10). Inertia is the within-cluster sum of squared distances: for every point, compute its squared Euclidean distance to its assigned centroid, then sum all these values across all clusters. As K increases, inertia always decreases — at K=N (each point is its own cluster), inertia is zero.

The 'elbow' is the point where the rate of inertia decrease changes from steep to shallow — where adding another cluster provides diminishing marginal improvement in compactness. This is chosen as the optimal K. For Model B, the elbow was observed at K=4, supporting our choice.

Q3. What is the difference between MAE and RMSE? When would you prefer one over the other?

MAE (Mean Absolute Error) is the average of absolute differences between actual and predicted values. Every error contributes equally regardless of its size. RMSE (Root Mean Squared Error) squares errors before averaging, then takes the square root — this means large errors are penalized disproportionately more than small ones.

You prefer MAE when outliers should not dominate the evaluation — for example, if a single exceptional disease outbreak week is not representative of normal operations. You prefer RMSE when large prediction errors are especially costly — for example, if a severe underestimate by 40 visits is much worse than two underestimates by 20 visits each. For clinic planning, RMSE is arguably more relevant because a severe stockout from one very wrong prediction could be dangerous.

Q4. What is Random Forest and why does it outperform a single Decision Tree?

A Decision Tree splits data based on feature thresholds to minimize a loss criterion (e.g., MSE for regression). A single tree is highly prone to overfitting — it memorizes the training data structure including noise. If trained on slightly different data, it produces very different splits (high variance).

Random Forest addresses this by training many trees (the forest), where each tree is trained on a bootstrap sample of the data AND only considers a random subset of features at each split. The final prediction is the average across all trees. Because each tree makes different errors (due to randomness), averaging cancels out individual errors — this is called variance reduction through ensembling. The result is a model that generalizes far better than any individual tree.

Q5. What is cross-validation and why is regular k-fold invalid for time-series data?

Cross-validation repeatedly splits data into train and test sets to get a more reliable performance estimate than a single split. Standard k-fold shuffles the data randomly and creates k equally sized folds, training on k-1 and testing on the remaining 1, cycling through all folds.

For time-series data, this is invalid because shuffling destroys temporal ordering. If Fold 3 (which temporally occurs in month 6) is used for training and Fold 1 (month 2) for testing, the model is trained on the future to predict the past — look-ahead leakage. `TimeSeriesSplit` prevents this by always training on the past and testing on the future, respecting the arrow of time.

Q6. What does it mean for K-Means to 'converge', and is convergence guaranteed?

K-Means converges when no data point changes its cluster assignment from one iteration to the next — the centroids stop moving. At this point, further iterations produce identical results, so the algorithm stops.

Convergence to some solution is guaranteed — K-Means will always terminate. However, it is NOT guaranteed to converge to the global optimum. Because initial centroid positions are random, K-Means can converge to a local minimum where clusters are suboptimal. This is why K-Means is typically run multiple times with different random initializations (controlled by `random_state=42` in scikit-learn) and the best result (lowest inertia) is kept.

Q7. What is feature engineering, and why was it necessary for the forecasting model?

Feature engineering is the process of creating new input variables from raw data that better capture the patterns the model needs to learn. Raw data alone (a date column and a visit count) cannot be directly fed into a regression model in a way that captures temporal structure.

For `UniMediTrend`, feature engineering created: lag features (converting the temporal sequence into usable numeric inputs), a rolling mean (capturing trend signal), and cyclical encodings (converting month/week into a continuous representation that correctly handles year-end wrap-around). Without these engineered features, the regression model would have almost no useful signal — it would essentially be predicting visit counts from just a date number, which carries no inherent demand information.

Q8. What is the difference between a model parameter and a hyperparameter?

A model parameter is a value learned automatically from data during training. Examples: the regression coefficients (beta values) in Linear Regression, the centroid positions in K-Means, and the tree split thresholds in Random Forest. You never set these manually — the algorithm finds optimal values.

A hyperparameter is a setting that controls the learning process itself, set by the user before training begins. Examples: K (number of clusters in K-Means), the number of trees in Random Forest (`n_estimators`), the learning rate in XGBoost, and the regularization strength in Ridge Regression. Hyperparameters are chosen via techniques like grid search or cross-validation, not learned from the training data directly.

B3. Tricky / Examiner-Favourite Questions

Q1. Random Forest is more powerful than Linear Regression. Why did Linear Regression perform BETTER on your holdout set?

Our dataset had only 79 weekly rows after aggregation. Random Forest, despite its ensemble design, is a high-variance model — it needs sufficient training data to generalize well. With so few rows, Random Forest overfit to the training data's specific noise patterns and failed to generalize to the holdout set.

Linear Regression has lower variance — it produces a simpler, smoother fit that is less sensitive to individual data points. On a small, noisy 79-row dataset, this simplicity was actually an advantage. This phenomenon is called the bias-variance tradeoff: in low-data regimes, higher-bias, lower-variance models often win. Random Forest would be expected to dominate with thousands of training rows.

Q2. If your R^2 is negative, why not just use the mean as your prediction? Isn't that provably better?

This is an excellent critical question. On the specific holdout test set in this experiment, yes — predicting the historical mean every week would have scored $R^2=0$, outperforming our models. However, this does not mean the mean is a useful predictor in practice.

Predicting the mean gives clinic managers no actionable information — it says 'expect the same as always' regardless of whether exams are approaching, a disease outbreak is occurring, or it is semester break. Our models, despite poor R^2 , at least attempt to incorporate recent trends via lag features and seasonal patterns. The appropriate next step is to add exogenous signals (exam calendar, weather) — not to abandon ML in favour of a naive constant. The negative R^2 is a diagnostic that the current feature set is insufficient, not that the approach is wrong.

Q3. Could you have used a neural network for this project? Why or why not?

Technically yes — a recurrent neural network (RNN) or Long Short-Term Memory (LSTM) network is specifically designed for sequential time-series data and could capture complex temporal dependencies that lag features approximate more crudely.

However, a neural network would be a poor choice for UniMediTrend for three reasons. First, we only have 79 training rows — neural networks are notoriously data-hungry and would massively overfit. Second, neural networks are opaque black boxes with no interpretability — for a healthcare planning tool where decisions affect staffing and medicine procurement, explainability to clinic management is critical. Third, the deployment complexity of a neural network is significantly higher than a serialized sklearn model. The project's goal was a deployable, auditable pipeline — neural networks are disproportionate to that goal at this scale.

Q4. Why is the silhouette score for Model B (0.1120) so much lower than Model A (0.3772)?

Model A clusters at the hostel level — 8 hostels described by aggregated diagnosis proportions. Aggregation smooths out individual visit noise, producing cleaner, more separable hostel profiles. The signal-to-noise ratio is higher when you are comparing 8 summarized profiles.

Model B clusters at the individual visit level — 5,799 records described by one-hot encoded diagnosis, severity, hostel, level, gender, and department. Individual visits are inherently noisier. A real student can have malaria AND stress-induced fatigue simultaneously, placing their visit profile on the boundary between Cluster 0 and Cluster 2. This genuine clinical overlap produces lower silhouette scores. It is a reflection of reality, not a modelling failure — the report correctly notes that moderate overlap 'indicates mixed-case real-world visit behaviour rather than perfectly separable categories'.

Q5. What would you change about the project if you had more time and real data?

With real data, the first priority would be integrating exogenous signals: the academic calendar (exam dates, semester start/end), weather data (rainfall and humidity correlate with malaria and waterborne illness incidence), and public health event records. These are the missing predictors that likely explain the negative R^2 .

On the modelling side, we would replace Random Forest with XGBoost or LightGBM — both mentioned in Future Work — with proper cross-validated hyperparameter tuning. We would add probabilistic forecasting (prediction intervals) using quantile regression or conformal prediction, giving the clinic a confidence range rather than a single point estimate. For clustering, we would test GMM to get soft probabilistic hostel assignments. Finally, we would implement drift monitoring — alerting when real incoming data diverges significantly from the training distribution, triggering a model retrain.

Q6. What is the curse of dimensionality and how did it affect your project?

The curse of dimensionality refers to the phenomenon where, as the number of dimensions (features) in a dataset increases, the data becomes increasingly sparse. In high-dimensional spaces, all points tend to become equidistant from each other — making distance-based metrics like Euclidean distance less meaningful.

In UniMediTrend, Model B applies K-Means to a one-hot encoded visit feature space with many columns (multiple diagnosis categories, multiple hostels, multiple departments). This high dimensionality partially explains the low silhouette score of 0.1120 — in high-dimensional space, K-Means struggles to find well-separated clusters because the concept of 'distance' degrades. This is one reason DBSCAN would also struggle in our Model B space — its density estimation is similarly affected. PCA could have been applied as a preprocessing step to reduce dimensions before clustering, which might have improved separation.

Q7. Why is idempotency important in your MongoDB pipeline design?

Idempotency means running the same pipeline operation multiple times produces the same result — no duplicate records, no cascading errors. In a data engineering pipeline that may be rerun during development, debugging, or scheduled refreshes, non-idempotent operations can corrupt data.

For example, if the enrichment pipeline inserts cluster labels into the database without first checking for existing records, each re-run would add duplicate enriched documents. The Raw vs Enriched collection separation ensures the raw data is never modified (immutable baseline), and the enrichment process can safely overwrite or upsert enriched records on each run. This design is critical for reproducibility — a core principle of auditable AI systems.

